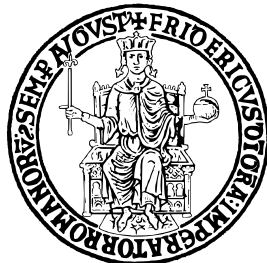


UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II



SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE

CORSO DI LAUREA MAGISTRALE IN INFORMATICA

DOCUMENTAZIONE DEL PROGETTO DI OPERATING SYSTEMS FOR MOBILE, CLOUD AND IoT

Docente

Prof. Porfirio Tramontana

Studenti

Salvatore Di Gennaro N97000479

Paolo Cammardella N86003043

Rodolfo Diana N97000508

Anno Accademico 2024/2025

Indice

1	Introduzione	2
1.1	Issues da risolvere	3
1.1.1	Aggiornamento dei <i>Tasks</i> asincroni	3
1.1.2	Migrazione delle informazioni	3
2	Sviluppo	5
2.1	Firebase	6
2.1.1	Struttura dati	6
2.1.2	Interfaccia grafica	7
2.2	Applicazione	8
2.2.1	Persistenza dati	8
2.2.2	Modifiche varie	9
2.2.3	Modifiche future	11
2.2.4	Diagrammi di classe	11
3	Analisi	14
3.1	Tempi di avvio	15
3.2	Utilizzo di memoria	16
4	Guida all'installazione	18
4.1	Importazione del database nella propria console Firebase	19
4.2	Android app	21
4.3	UnoProcidaStudenteTool	21
4.3.1	Recuperare il file di configurazione	22
4.3.2	Avvio del tool	23
5	Conclusioni	24
6	Appendice	26
6.1	Recupero dati	27
6.2	Struttura dati	28
6.2.1	transports	28
6.2.2	alerts	28
6.2.3	companies	29
6.2.4	taxis	29

1

Introduzione

Uno Procida Residente è un'applicazione open source per dispositivi Android che consente di visualizzare gli orari dei traghetti disponibili nel golfo di Napoli che partano da o arrivino all'isola di Procida, fornendo anche indicazioni su possibili ritardi o cancellazioni tramite segnalazioni degli altri utenti.¹ La presente documentazione illustra le modifiche apportate all'applicazione al fine di risolvere alcuni problemi presenti al suo interno, con l'obiettivo di aggiornare determinate funzionalità utilizzando le tecnologie più recenti e avanzate disponibili nell'ecosistema Android.

1.1 Issues da risolvere

1.1.1 Aggiornamento dei *Tasks* asincroni

Il primo *issue* affrontato è il seguente:

*L'applicazione si basa su AsyncTask, soluzione ormai considerata obsoleta e quindi deprecata. Gli AsyncTask servono ad avviare in parallelo, all'apertura dell'applicazione, la lettura degli aggiornamenti degli orari da file, la lettura dei dati meteo aggiornati, la lettura del file con le segnalazioni fatte dagli utenti.*²

L'utilizzo di una funzionalità deprecata rappresenta un rischio per la manutenibilità e la stabilità del codice, oltre a limitare la compatibilità con le versioni più recenti del sistema operativo.

1.1.2 Migrazione delle informazioni

Il secondo *issue* affrontato è il seguente:

*L'applicazione fa uso di file esterni disponibili su server esterni non sicuri (ad es. http): gli orari provengono da un file http mentre le segnalazioni degli utente da un file di testo ospitato da altervista.*³

Attualmente, questi dati sono memorizzati in file *CSV*, una soluzione non ottimale per diversi motivi:

- **Fragilità:** Una piccola modifica o un errore nel file *CSV* potrebbe compromettere il funzionamento dell'applicazione.
- **Scalabilità:** La gestione di grandi volumi di dati diventa inefficiente con un file *CSV*.
- **Mancanza di struttura:** I file *CSV* non forniscono un meccanismo di validazione dei dati, aumentando il rischio di inconsistenze.

Per ovviare a questi problemi, è stata proposta la migrazione dei dati verso un sistema di gestione più robusto e strutturato, ovvero un database remoto come **Firestore Realtime Database**. Questa migrazione oltre a migliorare l'affidabilità, la manutenibilità e la scalabilità del sistema, permette di passare

¹<https://github.com/reverse-unina/UnoProcidaResidente>

²<https://github.com/reverse-unina/UnoProcidaResidente/issues/2>

³<https://github.com/reverse-unina/UnoProcidaResidente/issues/5>

dal semplice protocollo HTTP a quello HTTPS. Quest'ultimo è una versione più sicuro di HTTP, viene solitamente utilizzato per proteggere la trasmissione di dati sensibili.

2

Sviluppo

2.1 Firebase

Nell'ambito del progetto è stato adottato il *Firebase Realtime Database* per gestire i dati in tempo reale, garantendo la sincronizzazione immediata e l'accesso da più dispositivi.

Grazie alla sua struttura *NoSQL* basata su **JSON**, offre un'elevata scalabilità e semplicità nella gestione delle informazioni, permettendo operazioni di lettura e scrittura efficienti. La scelta di *Firebase* è motivata dalla necessità di avere un sistema affidabile, accessibile e sempre aggiornato.

2.1.1 Struttura dati

Il database utilizza una struttura **JSON** gerarchica, organizzata nei seguenti nodi principali:

- **Transports:** Contiene informazioni dettagliate sui traghetti, inclusi orari di partenza e arrivo, porto di partenza e arrivo, nome della nave e prezzi per biglietti interi e ridotti. Ogni traghetto ha un identificativo univoco per un rapido accesso ai dati.
- **Alerts:** Memorizza segnalazioni degli utenti, registrando dettagli rilevanti come data, ora, posizione e natura del problema segnalato. Questo nodo è pensato per essere aggiornato dinamicamente e mantenere uno storico delle segnalazioni.
- **Companies:** Archivia i dettagli delle compagnie di trasporto, con informazioni sulle flotte, orari e servizi offerti. Questo consente un'integrazione più efficace tra gli operatori e il sistema centrale.
- **Taxis:** Contiene dati sui taxi disponibili, inclusi identificativi dei veicoli e contatti degli operatori. Questo permette un'organizzazione più strutturata e un accesso rapido ai servizi di trasporto locale.

Questa nuova organizzazione migliora l'efficienza nella gestione dei dati, ottimizza il recupero delle informazioni e supporta una migliore esperienza utente grazie alla rapidità di aggiornamento e accessibilità delle informazioni.

2.1.2 Interfaccia grafica

Per semplificare l'inserimento di nuove corse, è stata sviluppata un'interfaccia grafica esterna all'applicazione *Android*¹.

Questa interfaccia è stata progettata per facilitare l'immissione dei dati necessari, migliorando l'efficienza e l'usabilità del sistema.

The image shows a graphical user interface window titled "Uno Procida". It contains several input fields for data entry: "Mezzo di trasporto:" (dropdown), "Porto partenza:" (dropdown), "Porto arrivo:" (dropdown), "Ora partenza (hh:mm):" (text), "Ora arrivo (hh:mm):" (text), "Data inizio esclusione:" (text), "Data fine esclusione:" (text), "Prezzo intero:" (text), "Prezzo ridotto:" (text), and "Giorno della settimana:" (checkboxes for Lun, Mar, Mer, Gio, Ven, Sab, Dom). A "Salva" button is located at the bottom center.

Figura 2.1: Interfaccia grafica per l'inserimento dei dati.

L'applicazione è stata implementata in *Python*, sfruttando la libreria *tkinter* per la creazione dell'interfaccia grafica. La scelta di queste tecnologie è stata dettata dalla loro flessibilità e dalla capacità di consentire modifiche rapide, garantendo al contempo una bassa complessità del codice.

¹https://github.com/paoloCamardella/UnoProcidaResidente_Tool

2.2 Applicazione

In questa sezione verranno spiegate le modifiche apportate all'applicazione Android riguardo principalmente la rimozione degli AsyncTask deprecati, in favore di un'architettura più estendibile che consenta una manutenzione e ristrutturazione futura dell'interfaccia utente più semplice.

2.2.1 Persistenza dati

Gli AsyncTask sono stati sostituiti con classi che seguano il pattern *Data Access Object* (DAO), un pattern strutturale che consente di isolare il livello di logica dell'applicazione dal meccanismo di persistenza dei dati tramite un'API astratta. La complessità delle operazioni di archiviazione viene così nascosta, consentendo a entrambi gli strati di evolversi separatamente senza sapere nulla l'uno dell'altro con una conseguente maggiore modularità del codice.

Ovviamente non sarebbe ideale bloccare l'interfaccia fino a quando non venga ottenuta una risposta, pertanto tutte le operazioni devono essere asincrone ed è necessario gestire questa ulteriore complessità. Per avere una generalizzazione del meccanismo di callback, le diverse interfacce consentono l'accesso a degli oggetti *LiveData*: classi contenitore di dati a cui è possibile registrare degli osservatori che verranno notificati quando si verificano cambiamenti. Inoltre, tengono in considerazione il ciclo di vita degli ascoltatori e garantiscono che vengano chiamati solo quelli attivi.² Per avere ulteriori informazioni sullo stato delle operazioni sono stati creati degli oggetti *DataUpdate*, wrapper che possono contenere nuovi dati oppure eccezioni generate durante le richieste. In questo modo gli ascoltatori possono ricevere in modo semplice i risultati e reagire adeguatamente.

Utilizzando Firebase Realtime Database come struttura di back-end, è stato possibile utilizzare gli SDK messi a disposizione da Google per accedere direttamente al database in modo sicuro senza necessità di un server intermedio. Internamente, gestiscono un sistema di thread per eseguire le richieste e di cache per consentire l'accesso ai dati anche se il dispositivo è offline. La raccolta dati sulle previsioni meteo viene, invece, eseguita tramite richiesta HTTPS alle API di *OpenWeatherMap*³.

Migrazioni vecchi dati su Firebase

Nelle versioni precedenti del sistema, determinati tipi di informazioni, quali compagnie navali, taxi e prezzi delle corse, venivano recuperati direttamente da sezioni di codice in cui erano state scritte manualmente. Questo approccio, oltre a risultare poco scalabile e difficile da mantenere, comportava numerosi rischi di incoerenza e di errori, poiché i dati erano impossibili da aggiornare senza modificare il codice sorgente e ricompilare l'applicazione. Naturalmente, ciò non è accettabile per gli standard di flessibilità di un sistema moderno e si è deciso, pertanto, di spostare questi dati nel database remoto. Inoltre, sono stati creati appositi oggetti e classi per il recupero e la gestione di tali informazioni.

²<https://developer.android.com/topic/libraries/architecture/livedata>

³<https://openweathermap.org/api>

Aggiornamenti automatici

Per mantenere la compatibilità con le modalità d'uso attuali dell'applicazione, il recupero dei dati avviene ancora come nelle precedenti versioni una singola volta all'avvio oppure su richiesta manuale dell'utente. Le nuove classi DAO implementate necessitano che le procedure siano chiamate esplicitamente e l'utilizzo non risulta particolarmente diverso rispetto a prima. Tuttavia, il sistema è stato strutturato con un occhio di riguardo a possibili evoluzioni future, ipotizzando anche i casi in cui alcune funzionalità vengano sostituite con richieste periodiche per aggiornamenti in tempo reale, possibilmente anche con l'applicazione non attivamente usata dall'utente. La ricezione di segnalazioni e delle previsioni meteo rappresentano ottimi candidati per questi casi di utilizzo.

In quest'ottica, sono state approntate alcune implementazioni che sfruttano i *Service*⁴ di Android per eseguire operazioni in background e comunicazioni interprocessi (IPC) con Activity e Fragments. In particolare, questi servizi sono definiti *bound* perchè attivi in genere solo mentre servono un altro componente. È possibile avviarle come un qualsiasi altro service ed ottenere un oggetto *IBinder*⁵ tramite il quale è possibile interagire.

La classe *WeatherService* si occupa di ottenere aggiornamenti riguardanti le previsioni meteo eseguendo richieste periodiche alle API di OpenWeatherMap. Dato che i service vengono avviati nello stesso thread del componente chiamante, per evitare di bloccare l'interfaccia utente le richieste vengono eseguite tramite il package *java.util.concurrent* di Java 8, che fornisce una semplice interfaccia standardizzata per l'esecuzione di programmi multithread.

La classe *AlertsService* sfrutta invece le librerie di Firebase per essere notificata riguardo nuove segnalazioni sulle tratte: quando si verifica un aggiornamento al database vengono inviati i nuovi dati a tutti i dispositivi connessi.

Si ricorda che, allo stato attuale, queste classi appena descritte non sono utilizzate all'interno dell'applicazione e rappresentano solo un *proof of concept* di alcune possibili implementazioni, le quali dipenderanno fortemente dai futuri cambiamenti all'interfaccia utente dell'applicazione.

2.2.2 Modifiche varie

In concomitanza alla ristrutturazione dello strato di persistenza, sono stati effettuati dei cambiamenti più o meno necessari in altre sezioni dell'applicazione al fine di semplificare e rendere più mantenibile il codice.

Strutture dati

Le entità principali dell'applicazione hanno subito delle modifiche, dove era possibile farlo senza sconvolgere la compatibilità:

- è stata cambiata la visibilità dei campi pubblici introducendo i rispettivi getter e setter;

⁴<https://developer.android.com/develop/background-work/services/bound-services>

⁵<https://developer.android.com/reference/android/os/IBinder>

- metodi che gestivano lo stato dell'applicazione e non strettamente collegati alle entità sono stati rimossi dal loro interno;
- è stata introdotta una classe *Alert* per rappresentare le segnalazioni;
- la classe *Compagnia* è stata dotata di più metodi per migliorare l'accesso ai contatti;
- le strutture temporali della classe *Mezzo* sono state sostituite con quelle di *java.time* di Java 8 e i costi non sono più calcolati tramite formule *hard-coded*;
- è stata semplificata la classe *Osservazione* rimuovendo i metodi relativi all'ottenimento di stringhe.

Si ritiene che, se strutturate correttamente, si possa ottenere una migliore pulizia generale del codice.

Semplificazione activity

Un problema evidente dell'applicazione era l'alto accoppiamento tra le diverse classi: le modifiche alla UI erano spesso eseguite stesso all'interno degli Async-Task e l'asincronismo delle operazioni ha portato ad avere un alto numero di variabili globali comunicanti tra di loro. Il lavoro svolto sullo strato di persistenza e sulle strutture dati hanno comportato un miglioramento della gestione, ed è stato possibile isolare quasi tutta la logica di aggiornamento dell'interfaccia. Dove possibile per il momento, sono stati rimossi i riferimenti diretti all'activity nelle altre componenti, utilizzati precedentemente per l'elaborazione e il recupero di stringhe dalle risorse del progetto. Ciò è considerata una pratica non consigliata nello sviluppo Android. L'analisi del codice ha, inoltre, riportato alcuni bug minori che sono stati rimossi.

È stato infine sostituito il Dialog che appariva dopo l'aggiornamento delle previsioni meteo con un Toast, reputato più semplice dal punto di vista implementativo e di utilizzo da parte dell'utente.

Analytics

In molte sezioni del progetto venivano eseguite ripetute segnalazioni di eventi a Google Analytics, un processo che comportava una scrittura ridondante di codice in vari punti dell'applicazione. Ciò rendeva non solo il codice più difficile da comprendere, ma causava anche numerosi avvisi da parte di Android Studio relativi all'uso improprio o obsoleto di determinati metodi, aumentando così il rischio di errori futuri. Per risolvere queste problematiche, è stata progettata e implementata una classe di facciata, che ha semplificato e centralizzato l'invio degli eventi di analytics. Questo approccio ha permesso di ridurre la duplicazione, migliorare la gestione degli avvisi e, al contempo, incapsulare in modo più chiaro e modulare il funzionamento effettivo delle segnalazioni.

Rimozione file non necessari dal repository

È stata, infine, eseguita una pulizia dei file non necessari dal tracking del repository, come build di output o impostazioni personali rigenerabili automaticamente da Android Studio.

2.2.3 Modifiche future

Come scritto precedentemente, si è preferito non eseguire ulteriori modifiche ma di seguito verranno riportate alcune possibili idee:

- sostituire l'utilizzo delle classi Calendar in favore dei più moderni *LocalTime*, *LocalDate*, ecc;
- cercare di rimuovere i controlli eseguiti su stringa;
- cambiare visibilità delle variabili pubbliche e minimizzare il passaggio di dati tra una classe e l'altra;
- uniformare il codice utilizzando i nomi in inglese.

2.2.4 Diagrammi di classe

Di seguito verranno mostrati i diagrammi di classi delle diverse componenti realizzate per lo strato di persistenza.

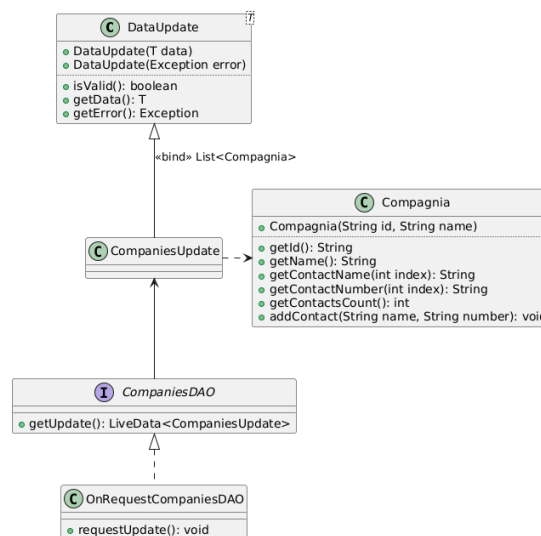


Figura 2.2: Diagramma compagnie

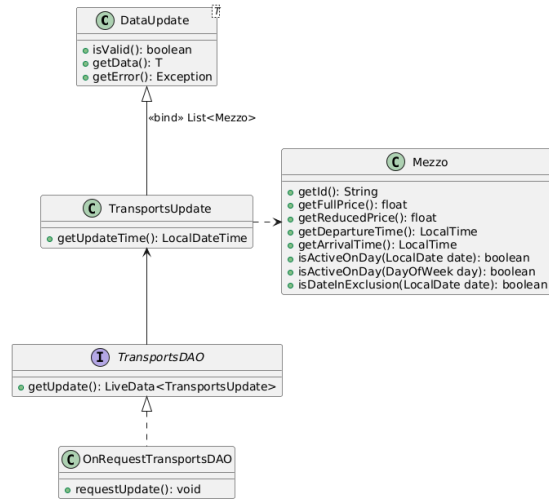


Figura 2.3: Diagramma corsa

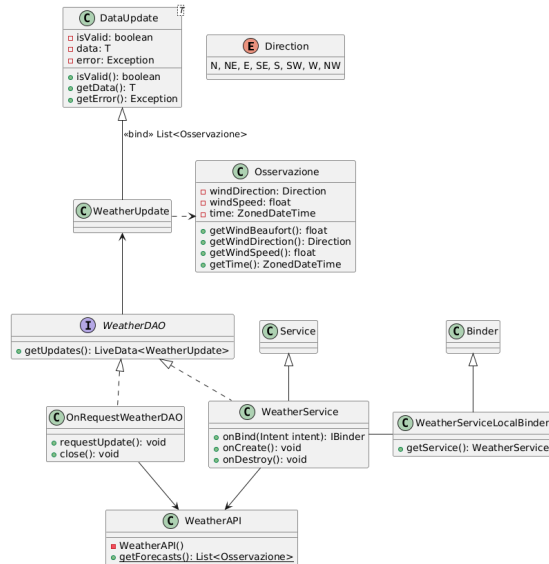


Figura 2.4: Diagramma previsioni meteo

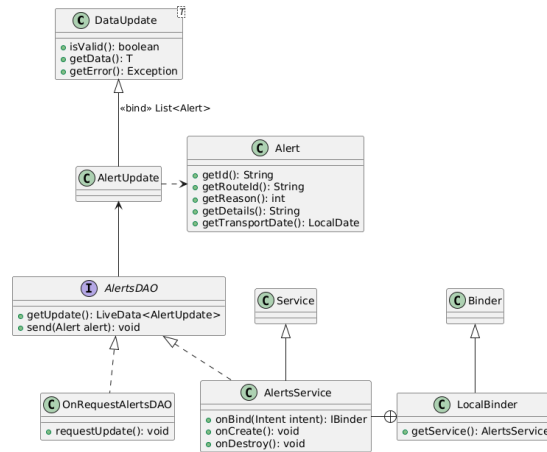


Figura 2.5: Diagramma segnalazioni

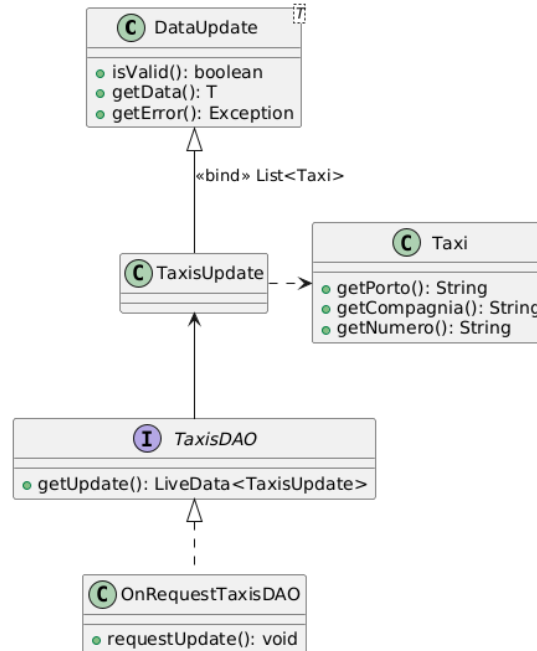


Figura 2.6: Diagramma taxi

3

Analisi

In questo capitolo saranno analizzate le prestazioni dell'applicazione, in particolare i *tempi di avvio* e *l'utilizzo della memoria*. Ciò è necessario per capire le modifiche apportate che benefici danno e valutare oggettivamente qualora queste modifiche siano necessarie e migliorative.

I test sono stati eseguiti con un dispositivo Pixel 9 API 35 emulato da Android Studio su un computer con le seguenti specifiche:

- processore AMD Ryzen 5 7600;
- scheda video NVIDIA GeForce GTX 1050 Ti;
- 32 GB di RAM DDR5;
- sistema operativo Linux Mint 22.1 Cinnamon.

3.1 Tempi di avvio

Gli utenti si aspettano che le app si carichino velocemente e siano reattive: un tempo di avvio lento non soddisfa questa aspettativa e può deluderli, fino a causare addirittura l'abbandono dell'applicazione. È importante, perciò, eseguire delle misurazioni per valutarne le prestazioni e gli eventuali cambiamenti da apportare.

Due metriche importanti per determinare l'avvio delle app sono il *time to initial display* (TTID) e il *time to full display* (TTFD)¹. Il TTID è il tempo che intercorre tra l'avvio del processo e l'istante in cui viene visualizzato il primo fotogramma dell'interfaccia utente. Il TTFD, invece, è il tempo necessario perché l'app diventi interattiva per l'utente, comprendendo anche il tempo impiegato per caricare i dati dalla rete o dal disco dopo la visualizzazione iniziale. Entrambi questi valori sono visualizzabili nei log delle applicazioni.

I test effettuati consistono nell'eseguire un determinato numero di volte il *cold start* sia della versione originale che della versione modificata del progetto, e misurarne entrambe le precedenti metriche per poi valutarne le differenze. Per cold start ci si riferisce all'avvio di un'app da zero, in cui il sistema deve ancora creare il processo per essa come quando viene avviata per la prima volta dopo l'accensione del dispositivo o da quando il sistema l'ha precedentemente fermata.

Il TTFD può ovviamente essere un punto variabile del codice che dipende da programma a programma, pertanto deve essere segnalato appositamente tramite il metodo `reportFullyDrawn()`. Nel caso in questione, tale metodo viene richiamato al termine della funzione `aggiornaLista()` nell'activity principale, che viene a sua volta chiamata quando sono stati caricati tutti i dati dalla rete. Nei risultati riportati di seguito è possibile notare come, grazie alle ottimizzazioni apportate, sia possibile riscontrare dei leggeri miglioramenti nelle metriche.

¹<https://developer.android.com/topic/performance/vitals/launch-time>

	Time to initial display (s)	Time to full display (s)
	0,946	1,006
	1,121	1,180
	1,031	1,113
	0,997	1,064
	1,018	1,077
	1,015	1,067
	1,015	1,073
	1,054	1,117
	1,176	1,226
	1,021	1,080
Media	1,039	1,100

Tabella 3.1: Tempi dell'applicazione originale

	Time to initial display (s)	Time to full display (s)
	0,830	1,065
	0,811	1,094
	0,765	0,986
	0,796	0,998
	0,785	0,996
	0,842	1,022
	0,775	1,016
	0,831	1,060
	0,793	0,986
	0,792	1,024
Media	0,802	1,025

Tabella 3.2: Tempi dell'applicazione modificata

3.2 Utilizzo di memoria

Per la misurazione dello spazio di memoria è stato utilizzato lo strumento di profiling *Live Telemetry* di Android Studio, che consente di visualizzare tramite grafico le metriche dell'applicazione in tempo reale.

Come è possibile vedere dai seguenti grafici, l'applicazione originale impiega inizialmente 46.6 MB di memoria, prima di assestarsi sui 24.5 MB in seguito alla prima run del garbage collector. In seguito alle modifiche applicate, lo spazio occupato inizialmente diventa 45.2 MB per poi trasformarsi stabilmente in 34.6 MB dopo l'esecuzione del garbage collector. Il maggiore spazio utilizzato è in linea con le aspettative derivanti dall'introduzione di più classi e oggetti.

Sebbene il *footprint* sia quindi aumentato, si ritiene che in termini assoluti l'incremento possa essere considerato marginale e un buon compromesso per i vantaggi derivanti dalla ristrutturazione del codice.

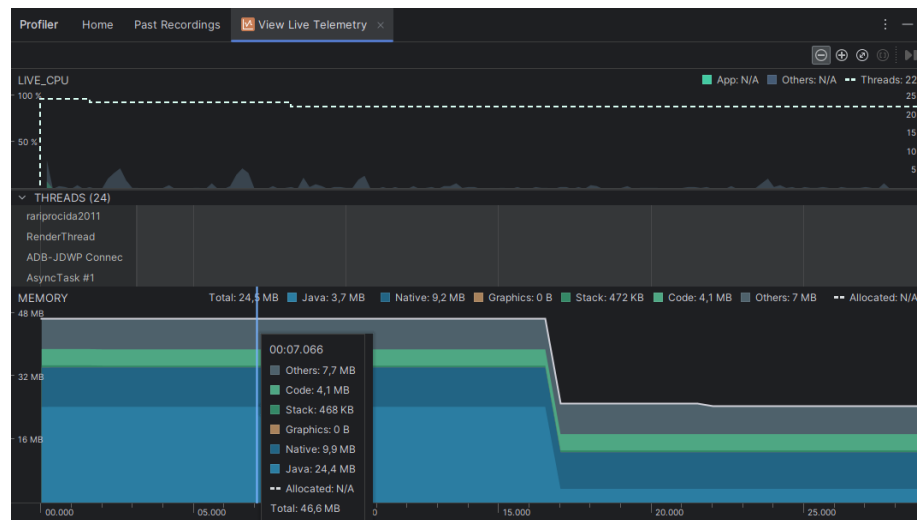


Figura 3.1: Utilizzo di memoria applicazione originale

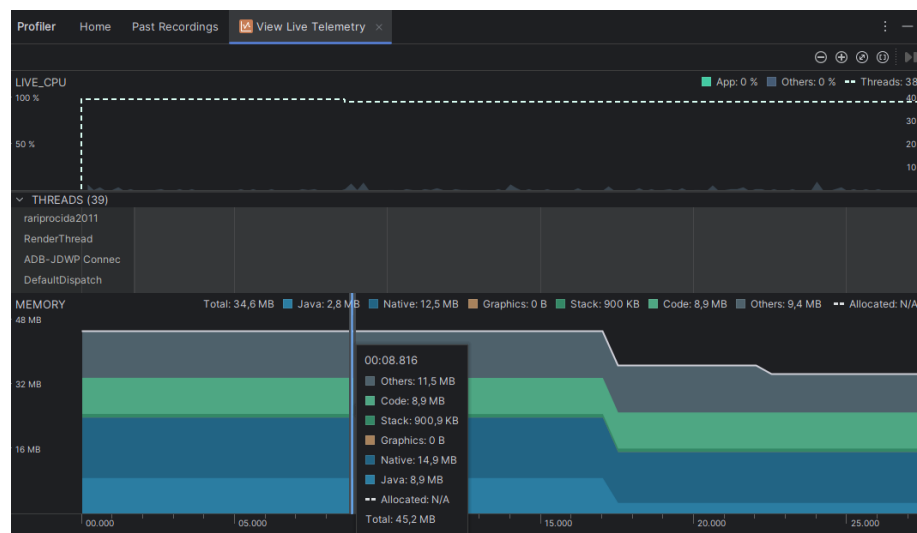


Figura 3.2: Utilizzo di memoria applicazione modificata

4

Guida all'installazione

Il presente capitolo ha lo scopo di fornire ai programmatori una guida chiara e dettagliata per proseguire nello sviluppo di questa applicazione. Questo risulta particolarmente necessario poiché le tecnologie impiegate non sono propriamente *plug & play* e richiedono una conoscenza approfondita per essere integrate e utilizzate correttamente.

4.1 Importazione del database nella propria console Firebase

Iniziamo con il creare un progetto su *Firebase*, dopo aver creato il proprio account, premere sul pulsante "Crea un nuovo progetto" ed inserisci i dati necessari.

Una volta all'interno della pagina del progetto, sulla sinistra ci sarà una barra laterale con un menù a tendina chiamato "*Creazione*"



Figura 4.1: Creazione progetto Firebase

All'interno del menu *Creazione*, clicca su *Realtime Database* e successivamente su "*Crea Database*", scegli le opzioni successive in base alle tue necessità.

All'interno del progetto è fornito un file denominato *db.json*, che rappresenta la struttura del database. Questo file, pur non essendo necessariamente aggiornato all'ultima versione, consente di eseguire test in ambiente locale.

Per importarlo nella propria console Firebase, è necessario seguire questi passaggi:

1. Accedere alla *console* di *Firebase*.
2. Navigare in **Realtime Database**.
3. Cliccare sui tre pallini (*menu delle opzioni*).
4. Selezionare l'opzione **Importa JSON**.

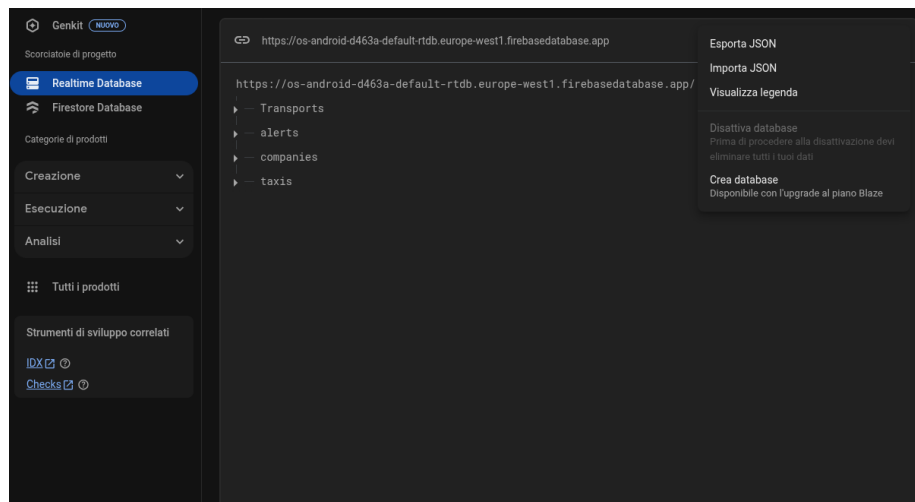


Figura 4.2: Importazione del database.

Una volta importato il database, recati nella sezione *Regole* nella console *Firebase* del *Realtime Database* e copia le regole all'interno del file "regole_Uno_Procida_residente_db.json" della repository del progetto.

4.2 Android app

Per integrare *Firebase Realtime Database* con l'applicazione *Android*, non basterà far altro che creare una key sulla *Console Firebase*, quindi navigare in **Impostazioni > Impostazioni progetto > Generali** e infine cliccare su **Aggiungi app**, come illustrato in Figura 4.3.

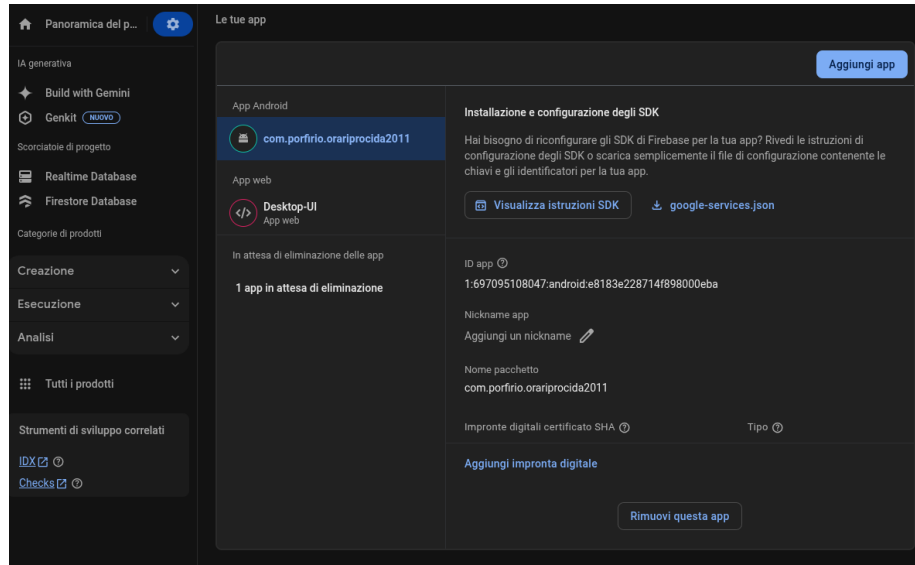


Figura 4.3: Pagina per generare la key di android

Il file `google-services.json`, è necessario per la comunicazione dell'applicazione *Android* con il *Firebase Realtime Database*, va sostituito con quello già presente nella repository all'interno della cartella `app/`.

4.3 UnoProcidaStudenteTool

La struttura del progetto del tool è illustrata in Figura 4.4. In particolare, il progetto contiene una cartella denominata `config`, all'interno della quale è necessario inserire un file chiamato `config.json` affinché il tool possa comunicare correttamente con il *Realtime Database* di *Firebase*.

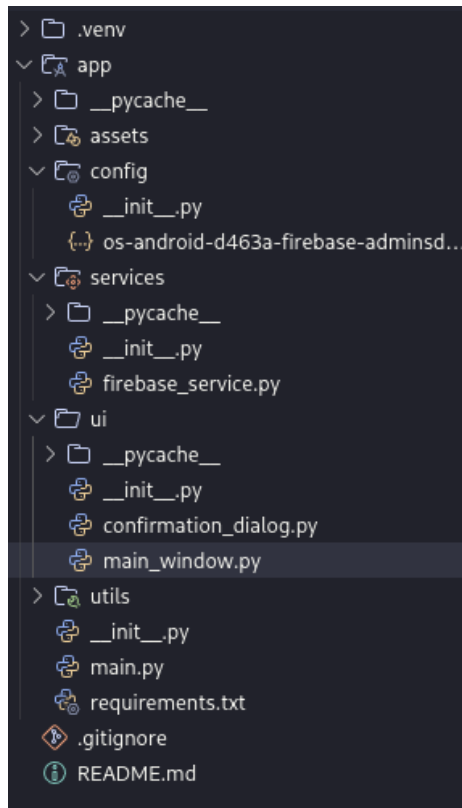


Figura 4.4: Struttura del tool

Il file di configurazione richiesto è quello che, in Figura 4.4, appare con il nome `os-android-d463a-firebase-adminsd....`

4.3.1 Recuperare il file di configurazione

Per ottenere il file `config.json`, è necessario accedere alla *console* di *Firebase*, quindi navigare in **Impostazioni > Impostazioni progetto > Account di servizio** e infine cliccare su **Genera nuova chiave privata**, come illustrato in Figura 4.5.

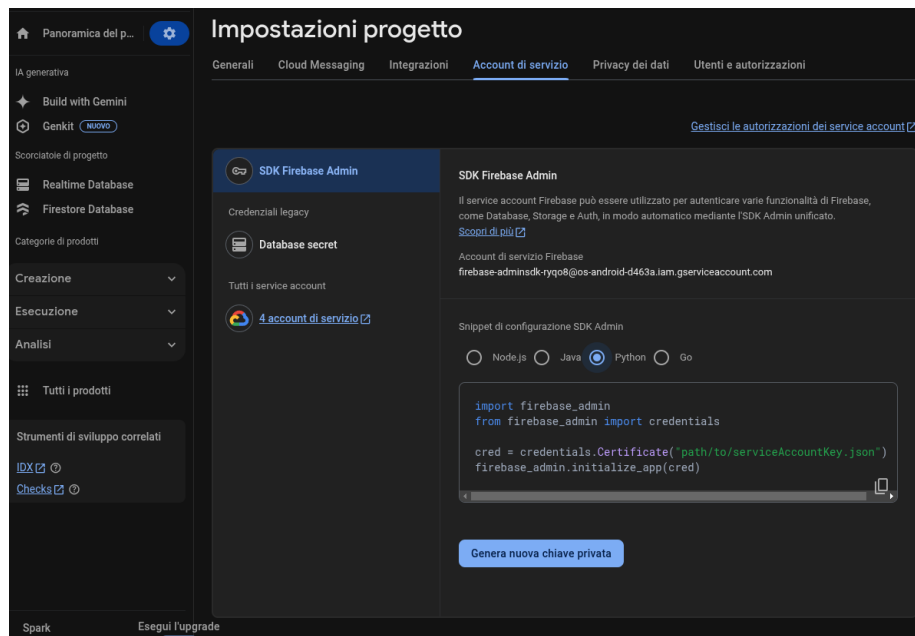


Figura 4.5: Recupero del file di configurazione

Una volta scaricato il file, sarà sufficiente inserirlo nella cartella `config` del progetto (Figura 4.4) per completare la configurazione.

4.3.2 Avvio del tool

Per avviare il tool, è possibile fare riferimento alla guida dettagliata presente nel file `README.md` del progetto.

All'interno del *README*, sono fornite istruzioni passo-passo per l'installazione, la configurazione e l'utilizzo del tool. Si consiglia di consultarlo attentamente prima di procedere con l'avvio, così da assicurarsi di seguire tutte le indicazioni necessarie per un corretto funzionamento del software.

5

Conclusioni

In conclusione, le modifiche apportate all'applicazione permettono di:

- **Migliorare la sicurezza:** L'implementazione di HTTPS garantisce che tutte le comunicazioni tra l'applicazione ed il *database* siano cifrate, proteggendo i dati sensibili da potenziali attacchi informatici.
- **Aumentare la scalabilità:** La riorganizzazione del codice in moduli e pacchetti ben definiti, facilita l'aggiunta di nuove funzionalità e la gestione di carichi di lavoro più elevati. L'integrazione con servizi esterni, come *Firebase*, consente di sfruttare infrastrutture *cloud* per una maggiore flessibilità e scalabilità.
- **Migliorare la manutenibilità:** L'adozione di determinate pratiche di sviluppo rende il codice più facile da mantenere e aggiornare, riducendo il rischio di errori e semplificando il debugging.
- **Garantire una migliore esperienza agli admin:** L'interfaccia grafica sviluppata per l'inserimento delle corse, rende l'aggiornamento degli orari dei traghetti e l'inserimento di nuovi più agevole, garantendo così una minore possibilità di errore ed una migliore consistenza dei dati.

In sintesi, queste migliorie non solo aumentano l'affidabilità e la sicurezza dell'applicazione, ma ne estendono anche le potenzialità, preparandola per future espansioni e adattamenti in un contesto tecnologico in continua evoluzione. Il lavoro svolto rappresenta un passo significativo verso un'applicazione più robusta, efficiente e pronta a soddisfare le esigenze degli utenti.

6

Appendice

6.1 Recupero dati

Tutte le operazioni di recupero all'interno dell'applicazione sono asincrone ed eseguite tramite oggetti DAO.

Tramite metodi `getUpdates()` è possibile accedere a oggetti LiveData ai quali registrarsi per ottenere le informazioni richieste. Le modalità di aggiornamento dipendono dalle implementazioni sottostanti dei DAO utilizzati, ma in generale tutte dovrebbero inviare i dati nel thread principale dell'applicazione. Al momento, sono necessarie delle richieste manuali tramite metodi appositi, come mostrato nel seguente esempio:

```
1 public class MyActivity extends Activity {
2     @Override
3     public void onCreate(Bundle savedInstanceState) {
4         // Do initialization and stuff
5
6         // Get the DAO object
7         // This one requires a manual request
8         OnRequestAlertsDAO alertsDAO = new
9         OnRequestAlertsDAO();
10
11         // Subscribe to future data updates
12         // It only needs to be done once
13         alertsDAO.getUpdates().observe(this, this::
14         onAlertsUpdate);
15
16         // Manually request an update
17         alertsDAO.requestUpdate();
18     }
19
20     @Override
21     public void onDestroy() {
22         // Do destroy and stuff
23
24         // Remove this observer
25         alertsDAO.getUpdates().removeObservers(this);
26     }
27
28     void onAlertsUpdate(AlertUpdate update) {
29         if (update.isValid()) {
30             List<Alert> alerts = update.getData();
31             // Process data
32         } else {
33             Exception e = update.getError();
34             // Handle exception
35         }
36     }
37 }
```

6.2 Struttura dati

Di seguito verranno elencate le strutture dati presenti nel database.

6.2.1 transports

```
1 "transports": [  
2   {  
3     "fineEsclusione": "2017-01-01",  
4     "giorniSettimana": "1234567",  
5     "inizioEsclusione": "2016-12-05",  
6     "nomeNave": "Aliscafo Caremar",  
7     "oraArrivo": "10:40:00",  
8     "oraPartenza": "10:15:00",  
9     "portoArrivo": "Procida",  
10    "portoPartenza": "Ischia Porto",  
11    "prezzoIntero": 10.0,  
12    "prezzoRidotto": 5.0  
13  }  
14 ]
```

6.2.2 alerts

```
1 "alerts": {  
2   "key": {  
3     "details": "string",  
4     "reason": 0,  
5     "routeId": "string",  
6     "transportDate": "1970-01-01"  
7   },  
8   ...  
9 }
```

6.2.3 companies

```
1 "companies": [  
2   {  
3     "name": "Caremar",  
4     "contacts": {  
5       "Ischia": [  
6         "0810000000",  
7         "0810000001"  
8       ],  
9       "Pozzuoli": [  
10        "0810000002",  
11        "0810000003"  
12      ]  
13    }  
14  }  
15 ]
```

6.2.4 taxis

```
1 "taxis": [  
2   {  
3     "location": "Procida",  
4     "name": "string",  
5     "number": "0810000000"  
6   }  
7 ]
```